



# **Dynamic Programming Project**

## **The Treasure Caravan**

**Roaa bzoor**

**12218040**

## Introduction:

The **0/1 Knapsack problem** is a classic optimization problem where we must choose items, each with a weight and a value, to maximize the total value without exceeding a weight limit. It is widely used to illustrate algorithmic techniques such as **Divide and Conquer** and **Dynamic Programming**, and here we will know the difference between them.

## Part1: Divide & Conquer

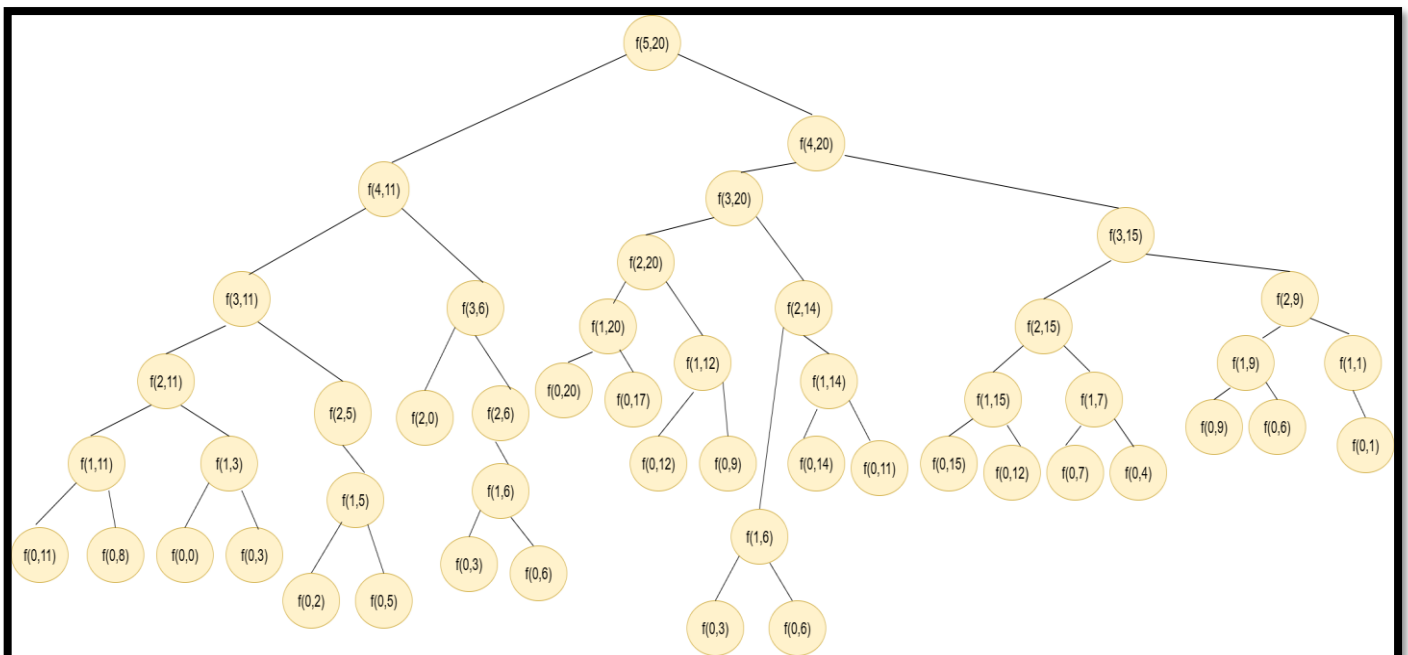
- 1- Define the value returned by the function  $f$  which we want to optimize.
- 2- Define the parameters which  $f$  depends on.
- 3- Draw the recursion tree for  $f$  using the values from the example above.
- 4- Write the recursive (divide and conquer) code to solve the question.

### 1. The value returned:

$\text{divide\_conquer}(i, w)$  returns the maximum total gold value that can be collected from the first  $i$  chests without exceeding a remaining weight  $w$ .

### 2. The function $\text{divide\_conquer}(i, w)$ where $i$ is the current chest and $w$ is the remaining weight capacity in the caravan.

### 3. The recursion tree, I use the draw.io to draw the tree.

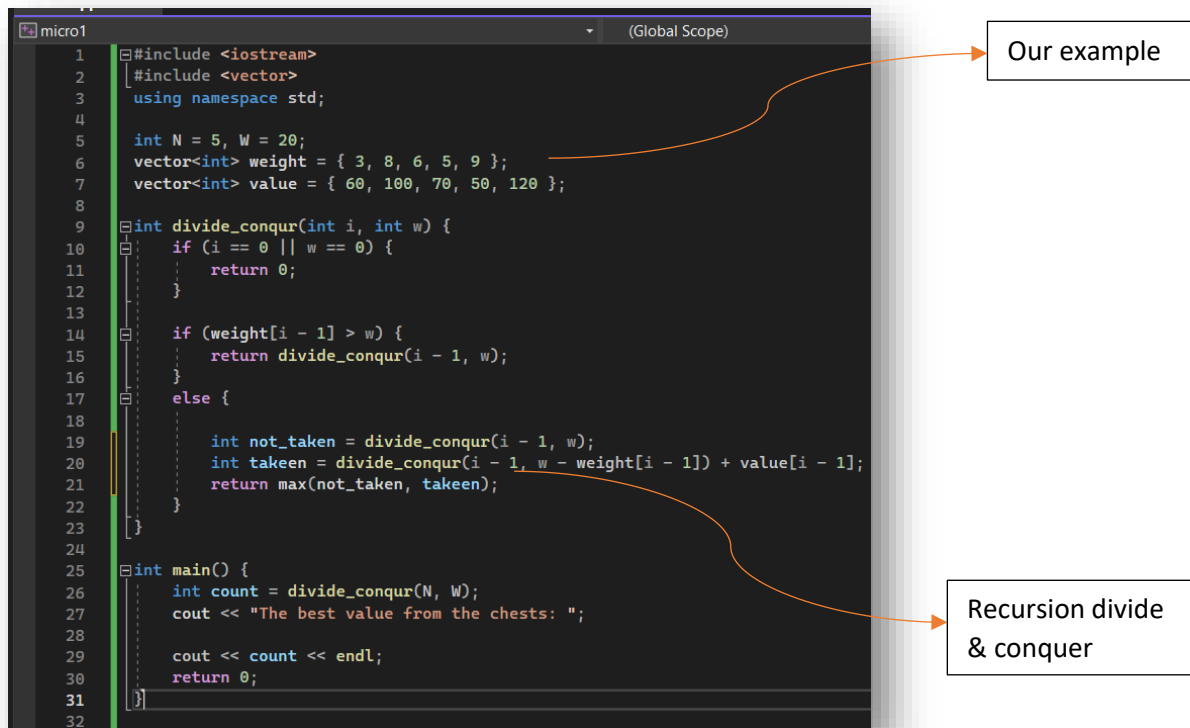


## Code explains:

In this method I used divide and conquer for the knapsack problem. At each chest I try two choices: take it or skip it, and then return the better result. This way the algorithm explores all combinations and gives the correct maximum value (280 in our example). The drawback is that it is slow for large inputs, so dynamic programming is better for bigger cases (don't take much time).

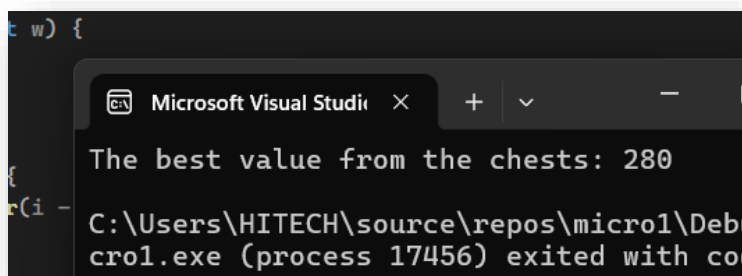
## Code for Divide & Conquer:

I do it for our example, I can do for loop and take the value from user.



```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 int N = 5, W = 20;
6 vector<int> weight = { 3, 8, 6, 5, 9 };
7 vector<int> value = { 60, 100, 70, 50, 120 };
8
9 int divide_conquer(int i, int w) {
10     if (i == 0 || w == 0) {
11         return 0;
12     }
13
14     if (weight[i - 1] > w) {
15         return divide_conquer(i - 1, w);
16     }
17     else {
18
19         int not_taken = divide_conquer(i - 1, w);
20         int takeen = divide_conquer(i - 1, w - weight[i - 1]) + value[i - 1];
21         return max(not_taken, takeen);
22     }
23 }
24
25 int main() {
26     int count = divide_conquer(N, W);
27     cout << "The best value from the chests: ";
28
29     cout << count << endl;
30     return 0;
31 }
```

## The result:



```
t w) {
{
r(i -

Microsoft Visual Studio
The best value from the chests: 280
C:\Users\HITECH\source\repos\micro1\Debug\micro1.exe (process 17456) exited with co
```

## Part2: Dynamic Programming

- 5- Draw the table and determine the dependencies between the table cells.
- 6- Determine the direction of movement within the table.
- 7- Write the Dynamic programming code which fills the table(s).
- 8- Write the code that will print the sequence of moves that go you the solution.

5. capacity →

Item

| i/<br>w | 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8   | 9   | 10  | 11  | 12  | 13  | 14  | 15  | 16  | 17  | 18  | 19  | 20  |
|---------|---|---|---|----|----|----|----|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 0       | 0 | 0 | 0 | 0  | 0  | 0  | 0  | 0  | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| 1       | 0 | 0 | 0 | 60 | 60 | 60 | 60 | 60 | 60  | 60  | 60  | 60  | 60  | 60  | 60  | 60  | 60  | 60  | 60  | 60  | 60  |
| 2       | 0 | 0 | 0 | 60 | 60 | 60 | 60 | 60 | 100 | 100 | 100 | 160 | 160 | 160 | 160 | 160 | 160 | 160 | 160 | 160 | 160 |
| 3       | 0 | 0 | 0 | 60 | 60 | 60 | 70 | 70 | 100 | 130 | 130 | 160 | 160 | 160 | 170 | 170 | 170 | 230 | 230 | 230 | 230 |
| 4       | 0 | 0 | 0 | 60 | 60 | 60 | 70 | 70 | 110 | 130 | 130 | 160 | 160 | 160 | 180 | 180 | 210 | 230 | 230 | 230 | 230 |
| 5       | 0 | 0 | 0 | 60 | 60 | 60 | 70 | 70 | 110 | 130 | 130 | 160 | 180 | 180 | 180 | 190 | 210 | 230 | 250 | 250 | 280 |

To determine the dependencies: other in tree obvious (taken →)

| i/<br>w | 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8   | 9   | 10  | 11  | 12  | 13  | 14  | 15  | 16  | 17  | 18  | 19  | 20  |
|---------|---|---|---|----|----|----|----|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 0       | 0 | 0 | 0 | 0  | 0  | 0  | 0  | 0  | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| 1       | 0 | 0 | 0 | 60 | 60 | 60 | 60 | 60 | 60  | 60  | 60  | 60  | 60  | 60  | 60  | 60  | 60  | 60  | 60  | 60  | 60  |
| 2       | 0 | 0 | 0 | 60 | 60 | 60 | 60 | 60 | 100 | 100 | 100 | 160 | 160 | 160 | 160 | 160 | 160 | 160 | 160 | 160 | 160 |
| 3       | 0 | 0 | 0 | 60 | 60 | 60 | 70 | 70 | 100 | 130 | 130 | 160 | 160 | 160 | 170 | 170 | 170 | 230 | 230 | 230 | 230 |
| 4       | 0 | 0 | 0 | 60 | 60 | 60 | 70 | 70 | 110 | 130 | 130 | 160 | 160 | 160 | 180 | 180 | 210 | 230 | 230 | 230 | 230 |
| 5       | 0 | 0 | 0 | 60 | 60 | 60 | 70 | 70 | 110 | 130 | 130 | 160 | 180 | 180 | 180 | 190 | 210 | 230 | 250 | 250 | 280 |

To show you some of cell's dependence.

6.

The dynamic programming table is filled row by row, starting from the base case at row  $i = 0$  and moving downward until the last row  $i = 5$ . Within each row, the filling proceeds from column  $w = 0$  to column  $w = 20$ . This means the overall direction of movement is from top to bottom across the rows and from left to right within each row, ensuring that all required subproblems are solved before being used in later computations.

- `chest_Value[j]` (Skip option)
- `chest_Value[j - wi]` (Take option).

**The code was submitted on hacker**

## The last requirement:

- print the sequence of moves that go you the solution

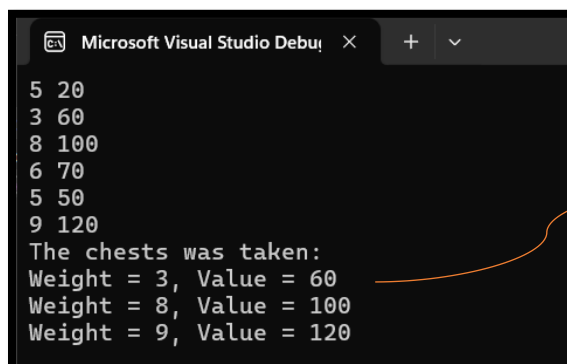


```
4
5  const int max_chest = 2025;
6  const int max_capacity = 2025;
7
8  int weight_heavy[max_chest];
9  int value_gold[max_chest];
10 int chest_value[max_capacity];
11 int valid[max_chest][max_capacity];
12
13 int treasure_chests(int chest, int capacity) {
14     for (int i = 0; i < chest; i++) {
15         int wi = weight_heavy[i];
16         int vi = value_gold[i];
17         for (int j = capacity; j >= wi; --j) {
18             if (chest_value[j - wi] + vi > chest_value[j]) {
19                 chest_value[j] = chest_value[j - wi] + vi;
20                 valid[i][j] = 1; // the chest taken
21             }
22         }
23     }
24     return chest_value[capacity];
25 }
26
27 void print_the_sequence(int chest, int capacity) {
28     int taken_weight[max_chest];
29     int taken_value[max_chest];
30     int count = 0;
31     int j = capacity;
32     for (int i = chest - 1; i >= 0; --i) {
33         if (valid[i][j]) {
34             taken_weight[count] = weight_heavy[i];
35             taken_value[count] = value_gold[i];
36             count++;
37             j -= weight_heavy[i]; // decrease the capacity
38         }
39     }
40     cout << "The chests was taken:\n";
41     for (int i = count - 1; i >= 0; --i) {
42         cout << "Weight = " << taken_weight[i]
43             << ", Value = " << taken_value[i] << "\n";
44     }
45 }
46
47 int main() {
48     int chest;
49     int capacity;
50     int count;
51
52     cin >> chest >> capacity;
53     for (int i = 0; i < chest; i++) {
54         cin >> weight_heavy[i] >> value_gold[i];
55     }
56
57     count = treasure_chests(chest, capacity);
58     print_the_sequence(chest, capacity);
59     cout << "\n";
60     //cout << "The best value from the chests: ";
61     // cout << count << "\n";
62     return 0;
63 }
```

Print code seq

Read the value from user.

## The output:



```
Microsoft Visual Studio Debu  X  +  v
5 20
3 60
8 100
6 70
5 50
9 120
The chests was taken:
Weight = 3, Value = 60
Weight = 8, Value = 100
Weight = 9, Value = 120
```

The sequence taken chest

## Code explains:

This code by Dynamic Programming, calculating maximum possible value of gold, and which chest are taken by their weight, this solution save time and effort to checking all possibilities it faster than divide & conquer.

## Conclusion

The **divide and conquer** solution for the **0/1 Knapsack problem** explores all possible combinations of items using recursion. While it guarantees correctness, its time complexity is exponential  $O(2^n)$ , making it impractical for larger inputs.

In contrast, the **dynamic programming** solution stores and reuses intermediate results in a table. This avoids repeated calculations and reduces the time complexity to polynomial  $O(N \times W)$ . As a result, dynamic programming is far more efficient and scalable, while divide and conquer is mainly useful for illustrating the problem's recursive structure.